

Video 10 – Neural Networks

Video Contents

• <i>Review of Lecture 9</i>	00:00 – 03:47
• <i>Introduction</i>	03:47 – 05:01
• <i>Batch Gradient Descent</i>	05:01 – 07:00
• <i>Stochastic Gradient Descent</i>	07:00 – 09:30
• <i>Benefits of Stochastic Gradient Descent</i>	09:30 – 14:32
• <i>An example of SGD</i>	14:32 – 19:58
• <i>The Neural Network Model</i>	19:58 – 36:36
• <i>Mathematical Model of Neural Networks</i>	36:36 – 43:40
• <i>Propagating back the error</i>	43:40 – 59:05
• <i>The Back-propagation algorithm</i>	59:05 – 1:05:12

Video Synopsis

Introduction

03:47 – 05:01

Neural networks are introduced as one of the key models in the revival of machine learning in recent decades. This lecture will deal with three core aspects:

1. Stochastic gradient descent as a more practical extension to gradient descent as introduced in lecture 9.
2. The neural network as a learning model
3. Back propagation as the learning algorithm generally used to train neural networks.

Batch Gradient Descent

05:01 – 07:00

Normal gradient descent as introduced in lecture 9 tries to minimise an error function that is defined as:

$$E_{in}(\mathbf{w}) = \frac{1}{N} \sum_{n=1}^N e(h(x_n), y_n)$$

and in lecture 9 we saw that for logistic regression:

$$e(h(x_n), y_n) = \ln(1 + e^{-y_n \mathbf{w}^T x_n})$$

By taking steps along the gradient of E_{in} but in the negative direction we can minimise this error (at least in a local sense) and this is the basis of gradient descent. So far we have considered an update in the direction $\Delta \mathbf{w} = -\eta \Delta E_{in}(\mathbf{w})$ based on calculating this update using all the training samples. One such update is called an epoch. Because for normal gradient descent this epoch contains the full batch of training samples, we also call it “batch gradient descent”.

Stochastic Gradient Descent

07:00 – 09:30

In stochastic gradient descent we do not consider the full batch, but we consider (initially) only one training sample at a time. As a result, we will introduce a certain randomness in the update of the weights: the average direction is still the same:

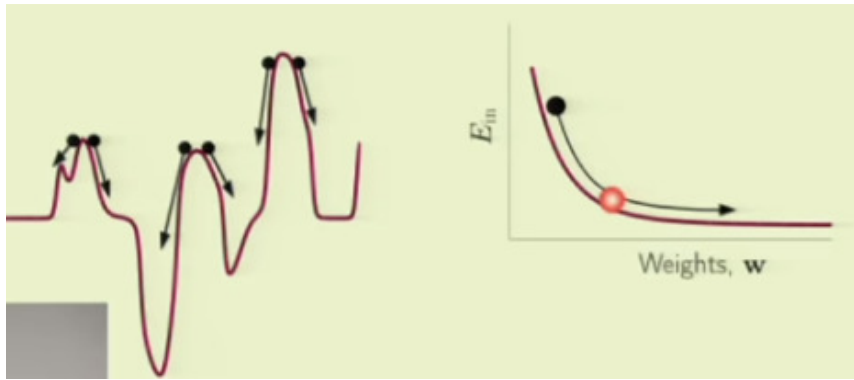
$$E[-\nabla e(h(x_n), y_n)] = \frac{1}{N} \sum_{n=1}^N -\nabla e(h(x_n), y_n) = -\nabla E_{in}$$

but every single update will introduce a certain deviation from this average. We will see that there are advantages to this stochastic aspect.

Benefits of Stochastic Gradient Descent (SGD)

09:30 – 14:32

1. The first benefit of SGD is the computational power required. Especially for large data sets normal gradient descent is very expensive as it needs to perform computations based on the full data set. This is largely resolved by using SGD.



2. The stochastic aspect in SGD helps in escaping from local minima. As shown in the picture, an error function will generally have multiple minima and only one of these can be the best one, i.e. the one where the error function is at a global

minimum. Normal gradient descent will find a minimum based on only descending on the error surface, which will likely lead to reaching a local minimum. Due to the randomness, SGD has a chance of escaping such a local minimum due to making a random step. Similarly, the stochastic aspect helps the network to move away from a region in which the error does no longer reduce.

3. SGD is a very simple algorithm as it works on one (or relatively few) samples at any one time. This simplicity results in a number of rules of thumb that generally work very well. For example in SGD, the rule of thumb to start with a learning rate η of 0.1 generally works very well (but of course it may be necessary to tune this parameter for better performance).

An example of SGD

14:32 – 19:58

Prof. Abu-Mostafa recalls the movie rating example from lecture 1. The winning submission in this computation used SGD. The approach takes the movie ratings and the user ratings as features (K each so 2K in total) in the following error function:

$$e_{ij} = \left(r_{ij} - \sum_{k=1}^K u_{ik} v_{jk} \right)^2$$

This error represents the difference between the rating of user i for movie j and the vector inner-product of user i 's profile vector u and the movie j 's characteristics vector v . By minimising this error for all examples available, the hope is that the user vector u will represent the user's interests as accurately as possible as well as that each movie is characterised as accurately as possible.

SGD was used to 'nudge' the 2K weights a little bit each time into the direction that minimised the error for that particular example. This was repeated for all examples thus resulting in overall very good performance.

The Neural Network Model

19:58 – 36:36

Neural networks were inspired by biological structures of neurons (computational units) with dendrites and synapses (the weights) connecting the individual neurons. The emerging understanding of brain function resulted in perceptrons in the early 20th century. To go beyond what perceptrons were able to do (with linear decision boundaries) we need to be able to perform operations that extend the capabilities of these simple networks. Three candidates are the logic AND, OR and NOT functions, and using appropriate weights, these can indeed be implemented using perceptrons in a layer type structure. Each layer consists of a number of nodes (generally drawn in a column) together with the input weights connection to these nodes. In this lecture we will only consider weights that feed forward towards the next layer and we will consider the problem of classification. Under these conditions, you can show that with just AND, OR and NOT logic gates implemented using perceptrons, pretty complex decision boundaries can be created. In general one can say that, the more neurons per layer and the more layers, the more complex the decision boundary can be. But based on the Theory of Generalization we also

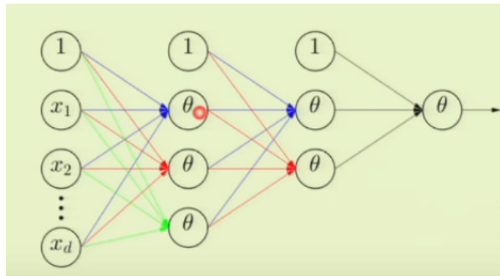
understand the implications this brings: 1) more neurons and layers results in a network with more parameters and this will result in a more complex model. As we now know, more complex models need more data to guarantee generalization. 2) the error surface (the multi-dimensional error as a function of the weight vector) that results from such a network, can be very complex and we will need a little bit more than just gradient descent to train the network successfully. The first thing we need to get rid off is the hard threshold (sign function) that we used in perceptrons as gradient descent can only be used once the error function is twice differentiable. Instead of using the logistic function used in logistic regression, we will use the tanh ('tench') or hyperbolic tangent as the activation function:

$$\tanh(s) = \frac{e^s - e^{-s}}{e^s + e^{-s}}$$

This function is appealing as its output varies from -1 to +1 (so we can still use it for AND, OR and NOT functionality) it is twice differentiable, it is symmetric around $s=0$ and it is also linear for small s . For larger s it becomes non-linear. The latter characteristics mean that the tanh can be used to capture linear as well as non-linear trends in data.

Mathematical Model of Neural Networks

36:36 – 43:40



As with linear regression and other models, we will call the free parameters the weights, with

$$w_{ij}^{(l)}$$

the weight from input i to neuron j for layer l . With these definitions we can define the input to neuron j in layer l as:

$$x_j^{(l)} = \theta(s_j^{(l)}) = \theta\left(\sum_{i=0}^{d^{(l-1)}} w_{ij}^{(l)} x_i^{(l-1)}\right)$$

where s is called the signal and $d^{(l-1)}$ is the number of neurons in layer $l-1$. This step is known as the forward pass: we apply this equation starting at the input layer and working towards the output to get the final output $x_1^{(L)}$ of our network.

Propagating back the error

43:40 – 59:05

To improve the ability of the forward pass to produce the correct output $x_1^{(L)}$, we now need to update the weights $w_{ij}^{(l)}$. We will use stochastic gradient descent for this, which means that we need to find the gradient $\nabla e(\mathbf{w})$ as the vector of all partial derivatives $\frac{\partial e(\mathbf{w})}{\partial w_{ij}^{(l)}}$.

Once we have found the gradient, we can update each of the weights as per the stochastic gradient principle: $\mathbf{w}(t+1) = \mathbf{w}(t) - \eta \nabla E_{in}$.

Using the chain rule we can write for each of the components of the gradient:

$$\frac{\partial e(\mathbf{w})}{\partial w_{ij}^{(l)}} = \frac{\partial e(\mathbf{w})}{\partial s_j^{(l)}} \frac{\partial s_j^{(l)}}{\partial w_{ij}^{(l)}}$$

To compute the gradient efficiently, we realise that we can write $x_j^{(l)} = \theta(s_j^{(l)})$ with:

$$s_j^{(l)} = \sum_{i=0}^{d^{(l-1)}} w_{ij}^{(l)} x_i^{(l-1)}$$

This equation shows that $\frac{\partial s_j^{(l)}}{\partial w_{ij}^{(l)}} = x_i^{(l-1)}$ which we already know from the forward pass. So what's left to be determined is the partial derivative: $\frac{\partial e(\mathbf{w})}{\partial s_j^{(l)}}$ which we will refer to as $\delta_j^{(l)}$ for brevity.

Hence, the components of the gradient can be written as:

$$\frac{\partial e(\mathbf{w})}{\partial w_{ij}^{(l)}} = \delta_j^{(l)} x_i^{(l-1)}$$

The δ 's can be calculated efficiently by starting at the output of the network and 'propagating' the results back to calculate the $\delta_j^{(l)}$ values for all other layers. We assume we have one output node which we will refer to as $x_1^{(L)}$. Hence the error at this the output is:

$$e(\mathbf{w}) = (x_1^{(L)} - y_n)^2 = (\theta(s_1^{(L)}) - y_n)^2$$

The derivative of the tanh function is:

$$\theta'(s) = 1 - \theta^2(s)$$

Hence, using the chain rule, we can calculate $\delta_1^{(L)}$:

$$\delta_1^{(L)} = \frac{\partial e(\mathbf{w})}{\partial s_1^{(L)}} = 2(\theta(s_1^{(L)}) - y_n)(1 - \theta^2(s_1^{(L)})) = 2(x_1^{(L)} - y_n)(1 - x_1^{(L)^2})$$

With the δ at the output of the network now known, we can 'propagate' back this quantity to all previous layers and compute the δ 's affecting the weight updates on those layers. For one neuron in a previous layer, we can say that the effect of the signal entering that neuron on the error $e(\mathbf{w})$ is:

$$\frac{\partial e(\mathbf{w})}{\partial s_i^{(l-1)}} = \frac{\partial e(\mathbf{w})}{\partial s_j^{(l)}} \frac{\partial s_j^{(l)}}{\partial x_i^{(l-1)}} \frac{\partial x_i^{(l-1)}}{\partial s_i^{(l-1)}}$$

However, this is not the full story. As there are $d^{(l)}$ neurons in the next layer, we should consider the effect the neuron in layer $l - 1$ on each of the neurons in layer l . Hence we get:

$$\delta_i^{(l-1)} = \sum_{j=1}^{d^{(l)}} \underbrace{\frac{\partial e(\mathbf{w})}{\partial s_j^{(l)}}}_1 \underbrace{\frac{\partial s_j^{(l)}}{\partial x_i^{(l-1)}}}_2 \underbrace{\frac{\partial x_i^{(l-1)}}{\partial s_i^{(l-1)}}}_3$$

Term 1: $\frac{\partial e(\mathbf{w})}{\partial s_j^{(l)}}$ is $\delta_j^{(l)}$ by definition.

Term 2: $\frac{\partial s_j^{(l)}}{\partial x_i^{(l-1)}}$ is the derivative of the signal towards one of its inputs which equals exactly the weight of that input: $w_{ij}^{(l)}$

Term 3: $\frac{\partial x_i^{(l-1)}}{\partial s_i^{(l-1)}}$ is the derivative of the output of the neuron towards the signal entering the neuron which we previously saw was $\theta'(s)$ evaluated at the value $s = s_i^{(l-1)}$.

Hence, $\delta_i^{(l-1)}$ can be written in terms of the already known quantities:

$$\delta_i^{(l-1)} = \sum_{j=1}^{d^{(l)}} \delta_j^{(l)} \cdot w_{ij}^l \cdot \theta'(s_i^{(l-1)})$$

As the argument of θ' is not reliant on the index j , it can be taken out of the sum:

$$\delta_i^{(l-1)} = \theta'(s_i^{(l-1)}) \sum_{j=1}^{d^{(l)}} \delta_j^{(l)} \cdot w_{ij}^l$$

which shows a very interesting symmetry between the forward and backpropagation pass:

The forward pass sums all inputs multiplied by their respective weights and applies the activation function (tanh in this case).

The backpropagation pass sums all deltas in the next layer multiplied by their respective weights and multiplies the result by the derivative of the activation function.

With the δ now known we can update the weights. Remember:

$$\mathbf{w}(t+1) = \mathbf{w}(t) - \eta \nabla E_{in}$$

If applied to each individual weight, we only need to consider the respective component of this gradient:

$$\frac{\partial e(\mathbf{w})}{\partial w_{ij}^{(l)}} = \delta_j^{(l)} x_i^{(l-1)}$$

So the update rule becomes:

$$w_{ij}^{(l)}(t+1) = w_{ij}^{(l)}(t) - \eta \frac{\partial e(\mathbf{w})}{\partial w_{ij}^{(l)}} = w_{ij}^{(l)}(t) - \eta \delta_j^{(l)} x_i^{(l-1)}$$

The Back-propagation algorithm

59:05 – 1:05:12

The back-propagation algorithm is summarised as a cycle of computing outputs based on inputs, followed by calculating delta's based on outputs. After each of these cycles, the weights can be updated based on the calculated outputs and deltas in each layer of the network. It is important not to initialise the weights in the network to 0 as then nothing would happen. This is due to the fact that the δ 's are calculated as a sum of outputs multiplied by the current weights. In general, you should choose initial weights that are small and random. As a final remark, the hidden layers in neural networks can be interpreted as engineering their own features out of the input data. However, there will not normally be a simple interpretation of these features. The networks ability to 'engineer' features based on 'looking' at the data is not an issue in terms of data snooping as we properly 'charge' the network by recognising that it has a higher VC dimension. As a rule of thumb (and very similar to the case for linear and logistic regression) we can assume that the VC dimension of a neural network is equal to the number of weights in the network.